ALLIANCE UNIVERSITY

Alliance School of Advanced Computing (ASAC)

A MICRO PROJECT REPORT

ON

"E-COMMERCE SALES ANALYSIS USING HADOOP AND SPARK"

(A Study on the UCI Online Retail Dataset)

Submitted in partial fulfilment of the requirements for
Big Data Analytics — Z1AMC 702

Submitted by
Mahi Satani
Registration No: 2511042210020

Under the Guidance of
Dr. Saravanan P

Alliance School of Advanced Computing (ASAC)
ALLIANCE UNIVERSITY, BENGALURU

1. Introduction

1.1 Background

Retail businesses generate large volumes of transactional data every day — invoices, product details, timestamps, prices, and customer identifiers. Traditional single-machine tools such as spreadsheets or a local Pandas script struggle once this data grows beyond a few hundred thousand rows, both in terms of memory and processing time. Big data frameworks such as the Hadoop Distributed File System (HDFS) and Apache Spark were built specifically to store and process datasets of this scale in a distributed, fault-tolerant, and parallelised manner.

This project applies that big data stack to a real transactional dataset — the UCI "Online Retail" dataset — to demonstrate a complete pipeline: distributed storage, distributed data cleaning and transformation, SQL-style business analytics, and visualization of the results.

1.2 Problem Statement

Given a raw, uncleaned e-commerce transaction log containing invalid entries (cancelled orders, missing customer identifiers, negative quantities), the project aims to build a reliable data pipeline that ingests this data into HDFS, cleans and transforms it using Spark, and extracts actionable business insights — including revenue trends, best-selling products, and customer value segmentation — that a retail business could use to guide inventory and marketing decisions.

1.3 Objectives

- To set up a single-node Hadoop (HDFS + YARN) and Apache Spark environment on WSL2 Ubuntu.
- To ingest the UCI Online Retail dataset into HDFS as the distributed storage layer.
- To clean and transform the raw data using PySpark DataFrame operations (removing cancellations, nulls, and invalid values).
- To perform business analytics using Spark SQL: revenue by country, monthly sales trend, best-selling products, order cancellation rate, and RFM-based customer segmentation.
- To persist analytical results back to HDFS in both CSV and Parquet formats.
- To visualize key results using Matplotlib and interpret them from a business perspective.

1.4 Scope

The project is limited to batch (not streaming) analysis of a single static dataset on a single-node pseudo-distributed cluster. It does not cover real-time ingestion,

multi-node cluster deployment, or machine learning model training, though these are discussed as future scope in Section 6.

1.5 Tools and Technologies Used

Layer	Technology	Purpose
Operating environment	WSL2 — Ubuntu 22.04	Linux environment for Hadoop/Spark on a Windows machine
Distributed storage	Hadoop 3.3.6 (HDFS)	Stores the raw dataset and all analysis output
Resource management	YARN	Cluster resource manager for Spark job execution
Processing engine	Apache Spark 3.5.7 (PySpark)	Distributed data cleaning, SQL analytics, RFM computation
Language	Python 3	PySpark job scripting
Visualization	Matplotlib	Chart generation from aggregated results
Dataset	UCI “Online Retail” dataset	541,909 transaction records, UK-based online retailer, Dec 2010–Dec 2011

2. System Analysis and Design

2.1 Existing System

Conventional retail sales analysis is typically carried out using spreadsheet software (Excel) or single-machine Python/Pandas scripts. These approaches load the entire dataset into the memory of one machine, which becomes slow or infeasible as data volume grows into millions of rows, and offer no built-in fault tolerance or horizontal scalability.

2.2 Proposed System

The proposed system distributes both storage and computation. The raw dataset is stored in HDFS, which splits and replicates data across the cluster's DataNodes. Apache Spark then reads this data as a distributed DataFrame and performs cleaning and aggregation operations in parallel across partitions, using an in-memory execution engine that is significantly faster than disk-based MapReduce. This architecture scales horizontally — the same code would run unchanged on a multi-node production cluster by simply changing the Spark master URL.

2.3 System Architecture

The pipeline consists of five stages, illustrated below: the raw CSV is loaded into HDFS, read and cleaned by PySpark, analysed with Spark SQL, written back to HDFS as CSV/Parquet, and finally visualised with Pandas and Matplotlib.

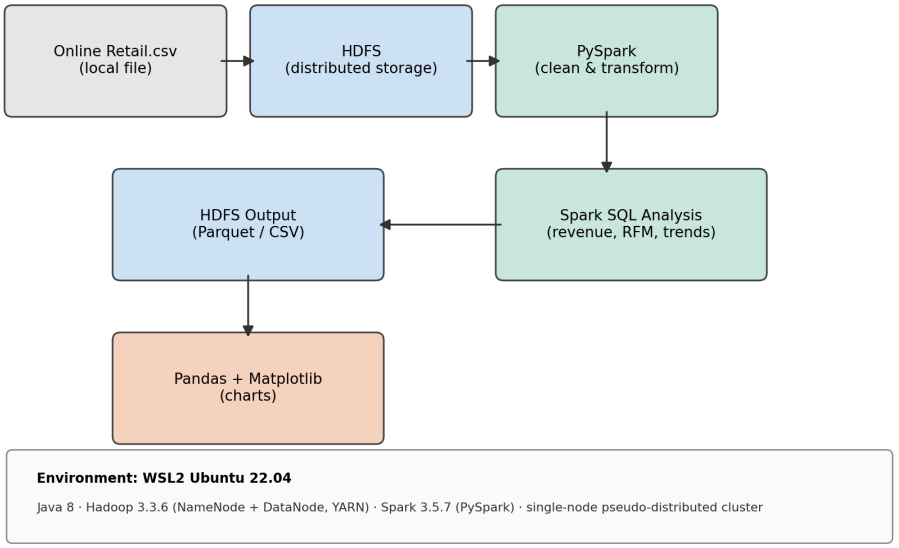

Figure 2.1: End-to-end pipeline architecture.

2.4 Module Description

2.4.1 Ingestion Module

Downloads the UCI Online Retail dataset, converts it from .xlsx to .csv, and loads it into HDFS under /user/<username>/ecommerce/input/ using the hdfs dfs -put command.

2.4.2 Data Cleaning Module

Reads the raw CSV from HDFS into a Spark DataFrame with an explicit schema, parses the InvoiceDate column into a proper timestamp, and filters out cancelled invoices (InvoiceNo starting with 'C'), rows with a null CustomerID, non-positive Quantity or UnitPrice, and duplicate rows. A derived TotalPrice column (Quantity × UnitPrice) is added for revenue calculations.

2.4.3 Analytics Module

Registers the cleaned DataFrame as a Spark SQL temporary view (sales) and runs six analytical queries: revenue by country, monthly sales trend, top 10 best-selling products, order cancellation rate, RFM customer segmentation, and top customers by lifetime spend.

2.4.4 Storage/Output Module

Writes every analysis result back to HDFS in both CSV (human-readable, for reporting) and Parquet (compressed, columnar, for reuse in tools like Hive) formats under /user/<username>/ecommerce/output/.

2.4.5 Visualization Module

Converts the small, already-aggregated Spark result sets into Pandas DataFrames and renders them as PNG charts using Matplotlib, saved to local disk for inclusion in this report.

2.5 Dataset Description

The UCI Online Retail dataset contains 541,909 transactional records from a UK-based online retailer between December 2010 and December 2011, with the following schema:

Column	Type	Description
InvoiceNo	String	6-digit invoice number; prefixed with 'C' if the transaction is a cancellation
StockCode	String	Unique product/item code
Description	String	Product name
Quantity	Integer	Quantity of the item purchased in that transaction
InvoiceDate	Timestamp	Date and time the transaction occurred
UnitPrice	Double	Price per unit, in GBP (£)
CustomerID	Integer	Unique 5-digit customer identifier (null for guest checkouts)
Country	String	Country of the customer

2.6 Hardware and Software Requirements

Requirement	Specification
Operating System	Windows 10/11 with WSL2 (Ubuntu 22.04)
RAM	Minimum 8 GB
Processor	Multi-core CPU (Spark used local[*] — 8 logical cores available)
Disk Space	Minimum 10 GB free (Hadoop, Spark, dataset, and output)
Java	OpenJDK 8 (1.8.0_492)
Hadoop	3.3.6 — single-node pseudo-distributed mode
Spark	3.5.7 (Scala 2.12.18), PySpark API
Python libraries	pyspark, pandas, matplotlib

3. IMPLEMENTATION

The full pipeline was implemented as a single PySpark job (ecommerce_analysis.py) submitted via spark-submit. The key stages of the implementation are described below with the corresponding code.

3.1 Spark Session Initialization

```
spark = (
    SparkSession.builder
    .appName("EcommerceSalesAnalysis")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate()
)
spark.sparkContext.setLogLevel("WARN")
```

The shuffle-partition count is lowered from Spark's default of 200 to 8, matching the small, single-node dataset size and avoiding unnecessary task overhead.

3.2 Reading Data from HDFS with an Explicit Schema

```
schema = StructType([
    StructField("InvoiceNo", StringType(), True),
    StructField("StockCode", StringType(), True),
    StructField("Description", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("InvoiceDate", StringType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("Country", StringType(), True),
])

raw_df = (
    spark.read
    .option("header", "true")
    .option("mode", "PERMISSIVE")
    .schema(schema)
    .csv(HDFS_INPUT_PATH)
)
```

An explicit schema is supplied instead of inferSchema=true, which avoids an extra full data scan and gives predictable column types.

3.3 Data Cleaning and Transformation

```
clean_df = (
    df.filter(~F.col("InvoiceNo").startswith("C"))
      .filter(F.col("CustomerID").isNotNull())
      .filter(F.col("Quantity") > 0)
      .filter(F.col("UnitPrice") > 0)
      .filter(F.col("Description").isNotNull())
      .withColumn("CustomerID", F.col("CustomerID").cast(IntegerType()))
      .withColumn("TotalPrice", F.round(F.col("Quantity") * F.col("UnitPrice"),
2))
      .dropDuplicates()
)
clean_df.cache()
clean_df.createOrReplaceTempView("sales")
```

The cleaned DataFrame is cached in memory and registered as a temporary view so all subsequent Spark SQL queries reuse it without recomputation (confirmed in the Storage tab of the Spark UI, Section 5).

3.4 Business Analysis with Spark SQL

Six analytical queries were run against the sales view. Two representative examples are shown below; the remainder follow the same groupBy/aggregate pattern.

```
revenue_by_country = spark.sql("""
    SELECT Country, ROUND(SUM(TotalPrice), 2) AS Revenue,
           COUNT(DISTINCT InvoiceNo) AS Orders
    FROM sales
    GROUP BY Country
    ORDER BY Revenue DESC
""")
```

Query 1: Total revenue and order count per country.

```
monthly_trend = spark.sql("""
    SELECT DATE_FORMAT(InvoiceDate, 'yyyy-MM') AS Month,
           ROUND(SUM(TotalPrice), 2) AS Revenue,
           COUNT(DISTINCT InvoiceNo) AS Orders
    FROM sales
    GROUP BY DATE_FORMAT(InvoiceDate, 'yyyy-MM')
    ORDER BY Month
""")
```

Query 2: Revenue and order count aggregated by calendar month.

3.5 RFM Customer Segmentation

Recency, Frequency, and Monetary (RFM) scores were computed per customer using Spark window functions, then combined into a business-friendly customer tier.

```
rfm = clean_df.groupBy("CustomerID").agg(
    F.datediff(snapshot_date, F.max("InvoiceDate")).alias("Recency"),
    F.countDistinct("InvoiceNo").alias("Frequency"),
    F.round(F.sum("TotalPrice"), 2).alias("Monetary"),
)

def add_score(dataframe, col, ascending, out_col):
    order_col = F.col(col).asc() if ascending else F.col(col).desc()
    w = Window.orderBy(order_col)
    return dataframe.withColumn(out_col, 6 - F.ntile(5).over(w))

rfm = add_score(rfm, "Recency", ascending=True, out_col="R_Score")
rfm = add_score(rfm, "Frequency", ascending=False, out_col="F_Score")
rfm = add_score(rfm, "Monetary", ascending=False, out_col="M_Score")

rfm = rfm.withColumn("RFM_Segment",
    F.concat_ws("", F.col("R_Score"), F.col("F_Score"), F.col("M_Score")))

rfm = rfm.withColumn("CustomerTier",
    F.when((F.col("R_Score") >= 4) & (F.col("F_Score") >= 4) &
(F.col("M_Score") >= 4), "Champion")
     .when((F.col("R_Score") >= 3) & (F.col("F_Score") >= 3), "Loyal")
     .when((F.col("R_Score") <= 2) & (F.col("F_Score") <= 2), "At Risk / Lost")
     .otherwise("Regular"))
```

F.ntile(5) splits customers into five equal-sized buckets (quintiles) per metric; the '6 -' inverts the bucket number so that 5 always means 'best'. Customers are then labelled into four business-readable tiers based on their combined scores.

3.6 Writing Results Back to HDFS

```
revenue_by_country.write.mode("overwrite").option("header", "true") \
    .csv(f"{HDFS_OUTPUT_PATH}/revenue_by_country_csv")
revenue_by_country.write.mode("overwrite") \
```

```
    .parquet(f"{HDFS_OUTPUT_PATH}/revenue_by_country_parquet")
```

Every result DataFrame is persisted in both CSV and Parquet formats using mode('overwrite'), so the job can be re-run idempotently during testing.

3.7 Job Submission

```
spark-submit --master local[*] --driver-memory 2g ecommerce_analysis.py
```

The job was submitted in local[*] mode, using all 8 logical cores of the WSL2 environment as Spark executor threads (verified in the Executors tab of the Spark UI, Section 5).

4. Results and Discussion

This section presents the output of every analysis, drawn directly from the CSV files written by the Spark job to HDFS (and copied locally for reporting).

4.1 Data Cleaning Summary

Metric	Value
Raw row count	541,909
Clean row count	392,692
Rows removed	149,217 (27.5%)

Roughly 27.5% of the raw records were discarded during cleaning. The dominant cause is missing CustomerID values (guest checkouts, which cannot be attributed to a specific customer for RFM analysis), followed by cancelled invoices and a small number of rows with non-positive quantity or price, which typically represent returns, adjustments, or data-entry artefacts rather than genuine sales.

4.2 Revenue by Country

The cleaned data spans 37 countries. The top 10 by revenue are shown below (source: revenue_by_country_csv).

Country	Revenue (£)	Orders
United Kingdom	7,285,024.64	16,646
Netherlands	285,446.34	94
EIRE	265,262.46	260
Germany	228,678.40	457
France	208,934.31	389
Australia	138,453.81	57
Spain	61,558.56	90
Switzerland	56,443.95	51
Belgium	41,196.34	98
Sweden	38,367.83	36

Table 4.1: Top 10 countries by revenue (full result set contains 37 countries).

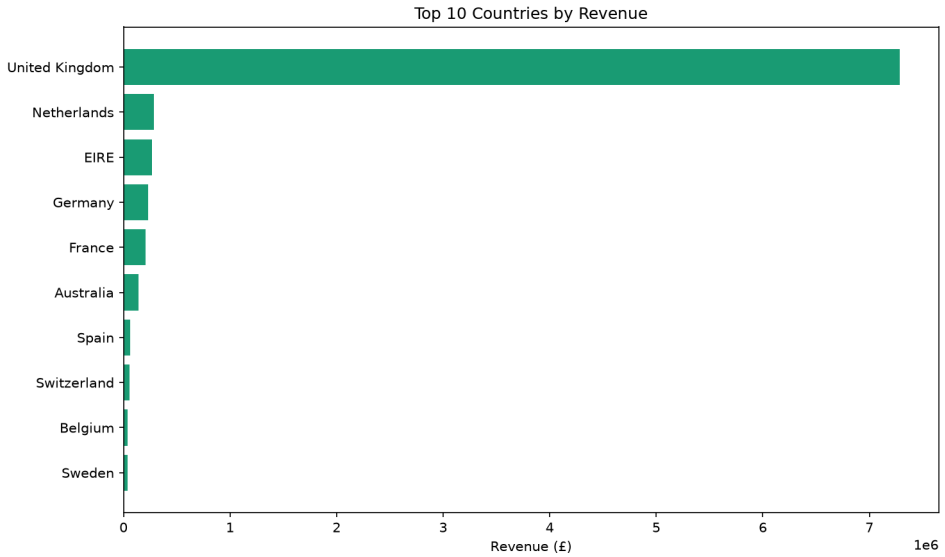

Figure 4.1: Top 10 countries by revenue.

The United Kingdom overwhelmingly dominates revenue (£7.29M, over 92% of total revenue), which is expected since the retailer is UK-based and the dataset skews heavily domestic. The Netherlands and EIRE (Ireland) are the largest export markets, but notably Germany and France generate more distinct orders (457 and 389) than the Netherlands (94) despite lower total revenue — suggesting smaller, more frequent orders in those markets versus a few very large bulk orders from Dutch customers. This is a useful signal for tailoring marketing spend by region.

4.3 Monthly Sales Trend

Month	Revenue (£)	Orders
2010-12	570,422.73	1,400
2011-01	568,101.31	987
2011-02	446,084.92	997
2011-03	594,081.76	1,321
2011-04	468,374.33	1,149
2011-05	677,355.15	1,555
2011-06	660,046.05	1,393
2011-07	598,962.90	1,331
2011-08	644,051.04	1,280
2011-09	950,690.20	1,755
2011-10	1,035,642.45	1,929
2011-11	1,156,205.61	2,657
2011-12	517,190.44	778

Table 4.2: Monthly revenue and order count (source: monthly_trend_csv).

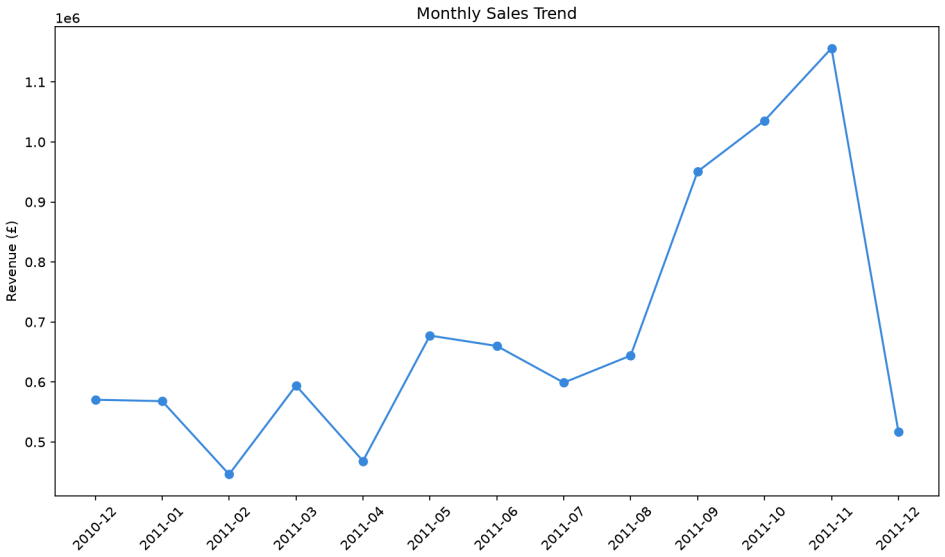

Figure 4.2: Monthly sales trend, December 2010 to December 2011.

Revenue is clearly seasonal: it climbs steadily from a February 2011 low (£446K) to a sharp peak in November 2011 (£1.16M, 2,657 orders), consistent with pre-Christmas holiday shopping. December 2011 shows a fall to £517K, but this reflects an incomplete month — the dataset ends on 9 December 2011 — rather than an actual decline. This pattern is strong evidence for building inventory and staffing plans around a Q4 demand surge.

4.4 Top 10 Best-Selling Products

Product	Qty Sold	Revenue (£)
PAPER CRAFT, LITTLE BIRDIE	80,995	168,469.60
MEDIUM CERAMIC TOP STORAGE JAR	77,916	81,416.73
WORLD WAR 2 GLIDERS ASSTD DESIGNS	54,319	13,558.41
JUMBO BAG RED RETROSPOT	46,078	85,040.54
WHITE HANGING HEART T-LIGHT HOLDER	36,706	100,392.10
ASSORTED COLOUR BIRD ORNAMENT	35,263	56,413.03
PACK OF 72 RETROSPOT CAKE CASES	33,670	16,381.88
POPCORN HOLDER	30,919	23,417.51
RABBIT NIGHT LIGHT	27,153	51,251.24
MINI PAINT SET VINTAGE	26,076	16,039.24

Table 4.3: Top 10 products by quantity sold (source: top_products_csv).

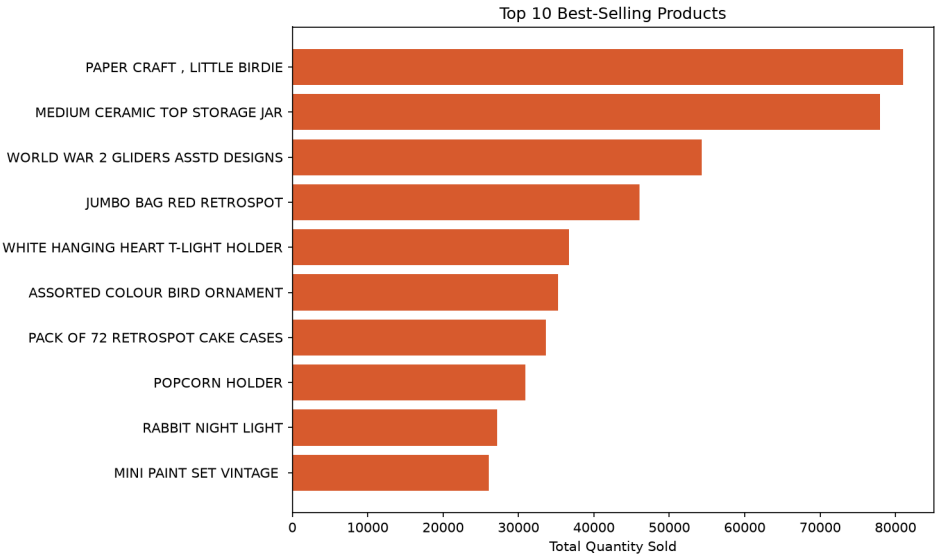

Figure 4.3: Top 10 best-selling products by quantity.

"Paper Craft, Little Birdie" leads by volume (80,995 units) and is also the single highest-revenue product (£168,469.60), while "World War 2 Gliders Asstd Designs" sells a high volume (54,319 units) but at very low revenue (£13,558.41) due to its low unit price — it is a high-turnover, low-margin item. This distinction between volume leaders and revenue leaders is important for inventory prioritisation: warehouse space should favour high-volume items, while promotional focus should favour high-revenue ones such as the "White Hanging Heart T-Light Holder" (£100,392.10 from a comparatively modest 36,706 units).

4.5 Order Cancellation Rate

Metric	Value
Total distinct invoices	25,900
Cancelled invoices	3,836
Cancellation rate	14.81%

Nearly 1 in 7 invoices (14.81%) is a cancellation. This is a materially high rate for a retail operation and points to potential issues worth investigating further — for example, whether cancellations cluster around specific products, customers, or time periods — which could inform returns-policy or checkout-flow improvements.

4.6 RFM Customer Segmentation

4,338 unique customers were scored on Recency, Frequency, and Monetary value and grouped into four tiers (source: rfm_csv).

Customer Tier	Number of Customers	% of Customers
At Risk / Lost	1,207	27.8%

Customer Tier	Number of Customers	% of Customers
Loyal	1,077	24.8%
Regular	1,054	24.3%
Champion	1,000	23.1%

Table 4.4: Customer distribution across RFM tiers (total 4,338 customers).

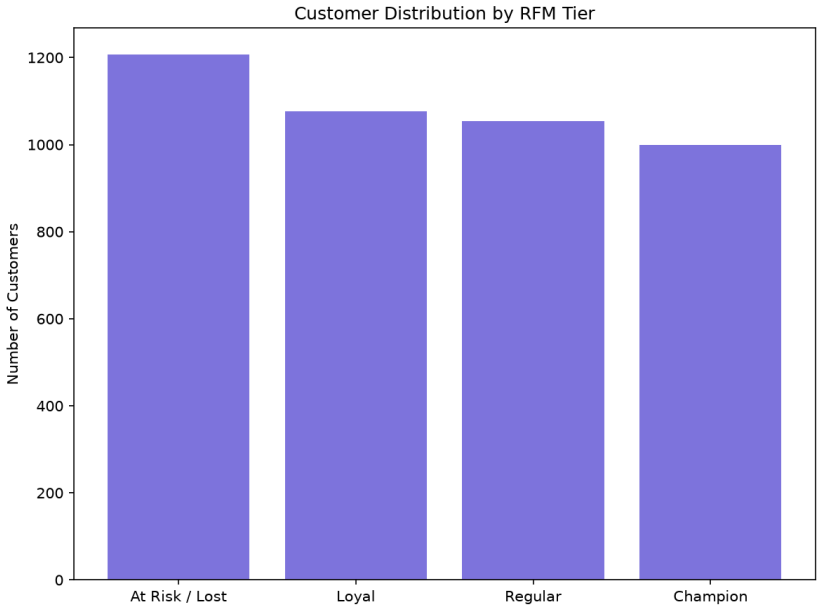

Figure 4.4: Customer distribution by RFM tier.

The customer base splits fairly evenly across tiers, but the largest single group — "At Risk / Lost" (27.8%) — is a warning sign: over a quarter of customers have both low recency and low frequency scores, meaning they have neither purchased recently nor often. Meanwhile, "Champions" (23.1%) represent the core loyal base driving disproportionate revenue and are the customers most worth retaining through loyalty programmes.

4.7 Top Customers by Lifetime Spend

CustomerID	Recency (days)	Frequency	Monetary (£)	Tier
14646	2	73	280,206.02	Champion
18102	1	60	259,657.30	Champion
17450	9	46	194,390.79	Champion
16446	1	2	168,472.50	Loyal
14911	2	201	143,711.17	Champion
12415	25	21	124,914.53	Champion
14156	10	55	117,210.08	Champion
17511	3	31	91,062.38	Champion
16029	39	63	80,850.84	Loyal

CustomerID	Recency (days)	Frequency	Monetary (£)	Tier
12346	326	1	77,183.60	At Risk / Lost

Table 4.5: Top 10 customers by lifetime spend (source: top_customers_csv).

Customer 14646 is the single highest-value customer (£280,206.02 across 73 orders, last purchase just 2 days before the snapshot date) and is correctly classified as a Champion. Two entries stand out as exceptions to the general pattern: customer 16446 spent £168,472.50 in only 2 orders — an extremely large average order value — and is scored "Loyal" rather than "Champion" purely because of low order frequency, illustrating a real limitation of frequency-based RFM scoring for bulk/wholesale buyers. Customer 12346 spent £77,183.60 in a single order 326 days before the snapshot date, correctly flagged "At Risk / Lost" despite high historical value — exactly the kind of high-value-but-dormant customer a win-back campaign should target.

4.8 Discussion Summary

- The business is heavily UK-concentrated but has healthy secondary markets in the Netherlands, Ireland, Germany, and France.
- Sales are strongly seasonal, peaking in November ahead of the Christmas period — a clear signal for inventory and staffing planning.
- Best-selling products by volume and by revenue are not always the same items, which has direct implications for warehousing versus marketing priorities.
- A cancellation rate of 14.81% is high enough to warrant a dedicated root-cause investigation.
- RFM segmentation surfaces a large "At Risk / Lost" segment (27.8%) alongside a strong Champion base (23.1%), giving the business two clear, actionable customer groups to target with different campaigns (win-back vs loyalty rewards).

5. Screenshots

This section documents the pipeline execution through terminal output and the Spark Web UI (http://localhost:4040), captured while the job was running.

5.1 Pipeline Execution — Terminal Output

5.1.1 Reading Raw Data

Figure 5.1: Raw row count after reading the CSV from HDFS — 541,909 records loaded.

5.1.2 Data Cleaning

```
======================================================================
STEP 2: Cleaning data
======================================================================
Clean row count: 392,692
Rows removed: 149,217 (27.5%)
```

Figure 5.2: Post-cleaning row count — 392,692 records retained, 149,217 (27.5%) removed.

5.1.3 Analysis 1 — Revenue by Country

```
======================================================================
ANALYSIS 1: Revenue by Country
======================================================================
+--------------+----------+------+
|Country       |Revenue   |Orders|
+--------------+----------+------+
|United Kingdom|7285024.64|16646 |
|Netherlands   |285446.34 |94    |
|EIRE          |265262.46 |260   |
|Germany       |228678.4  |457   |
|France        |208934.31 |389   |
|Australia     |138453.81 |57    |
|Spain         |61558.56  |90    |
|Switzerland   |56443.95  |51    |
|Belgium       |41196.34  |98    |
|Sweden        |38367.83  |36    |
+--------------+----------+------+
only showing top 10 rows
```

Figure 5.3: Console output of the revenue-by-country Spark SQL query (top 10 rows shown).

5.1.4 Analysis 2 — Monthly Sales Trend

```
======================================================================
ANALYSIS 2: Monthly Sales Trend
======================================================================
+-------+----------+------+
|Month  |Revenue   |Orders|
+-------+----------+------+
|2010-12|570422.73 |1400  |
|2011-01|568101.31 |987   |
|2011-02|446084.92 |997   |
|2011-03|594081.76 |1321  |
|2011-04|468374.33 |1149  |
|2011-05|677355.15 |1555  |
|2011-06|660046.05 |1393  |
|2011-07|598962.9  |1331  |
|2011-08|644051.04 |1280  |
|2011-09|950690.2  |1755  |
|2011-10|1035642.45|1929  |
|2011-11|1156205.61|2657  |
|2011-12|517190.44 |778   |
+-------+----------+------+
```

Figure 5.4: Console output of the monthly sales trend query, December 2010 to December 2011.

5.1.5 Analysis 3 — Top 10 Best-Selling Products

```
======================================================================
ANALYSIS 3: Top 10 Best-Selling Products (by quantity)
======================================================================
+--------------------------------+-------------+--------+
|Description                     |TotalQuantity|Revenue |
+--------------------------------+-------------+--------+
|PAPER CRAFT , LITTLE BIRDIE     |80995        |168469.6|
|MEDIUM CERAMIC TOP STORAGE JAR  |77916        |81416.73|
|WORLD WAR 2 GLIDERS ASSTD DESIGNS|54319       |13558.41|
|JUMBO BAG RED RETROSPOT         |46078        |85040.54|
|WHITE HANGING HEART T-LIGHT HOLDER|36706      |100392.1|
|ASSORTED COLOUR BIRD ORNAMENT   |35263        |56413.03|
|PACK OF 72 RETROSPOT CAKE CASES |33670        |16381.88|
|POPCORN HOLDER                  |30919        |23417.51|
|RABBIT NIGHT LIGHT              |27153        |51251.24|
|MINI PAINT SET VINTAGE          |26076        |16039.24|
+--------------------------------+-------------+--------+
```

Figure 5.5: Console output listing the top 10 products by quantity sold.

5.1.6 Analysis 4 — Order Cancellation Rate

```
======================================================================
ANALYSIS 4: Order Cancellation Rate
======================================================================
Total distinct invoices : 25,900
Cancelled invoices      : 3,836
Cancellation rate       : 14.81%
```

Figure 5.6: Console output of the cancellation-rate calculation — 14.81% of invoices cancelled.

5.1.7 Analysis 5 — RFM Customer Segmentation

```
======================================================================
ANALYSIS 5: RFM Customer Segmentation
======================================================================
```

Figure 5.7: Header for the RFM segmentation analysis stage.

```
+----------+-------+---------+---------+-------+-------+-------+-----------+--------------+
|CustomerID|Recency|Frequency|Monetary |R_Score|F_Score|M_Score|RFM_Segment|CustomerTier  |
+----------+-------+---------+---------+-------+-------+-------+-----------+--------------+
|14646     |2      |73       |280206.02|5      |5      |5      |555        |Champion      |
|18102     |1      |60       |259657.3 |5      |5      |5      |555        |Champion      |
|17450     |9      |46       |194390.79|5      |5      |5      |555        |Champion      |
|16446     |1      |2        |168472.5 |5      |3      |5      |535        |Loyal         |
|14911     |2      |201      |143711.17|5      |5      |5      |555        |Champion      |
|12415     |25     |21       |124914.53|4      |5      |5      |455        |Champion      |
|14156     |10     |55       |117210.08|5      |5      |5      |555        |Champion      |
|17511     |3      |31       |91062.38 |5      |5      |5      |555        |Champion      |
|16029     |39     |63       |80850.84 |3      |5      |5      |355        |Loyal         |
|12346     |326    |1        |77183.6  |1      |1      |5      |115        |At Risk / Lost|
+----------+-------+---------+---------+-------+-------+-------+-----------+--------------+
```

Figure 5.8: Top 10 customers ranked by monetary value, with computed R/F/M scores, RFM segment code, and customer tier.

5.1.8 Chart Generation and Pipeline Completion

```
======================================================================
STEP 9: Generating charts
======================================================================
```

Figure 5.9: Chart-generation stage beginning.

```
Charts saved to: /home/mahi/ecommerce-bigdata/charts

======================================================================
PIPELINE COMPLETE
Results in HDFS under: /user/mahi/ecommerce/output
Charts saved locally under: /home/mahi/ecommerce-bigdata/charts
======================================================================
```

Figure 5.10: Confirmation that charts were saved locally and all results were written to HDFS, marking successful pipeline completion.

5.2 Apache Spark Web UI (localhost:4040)

The Spark UI provides a live view of job execution, useful for demonstrating that the workload is genuinely running as a distributed Spark application rather than plain Python.

5.2.1 Jobs Tab

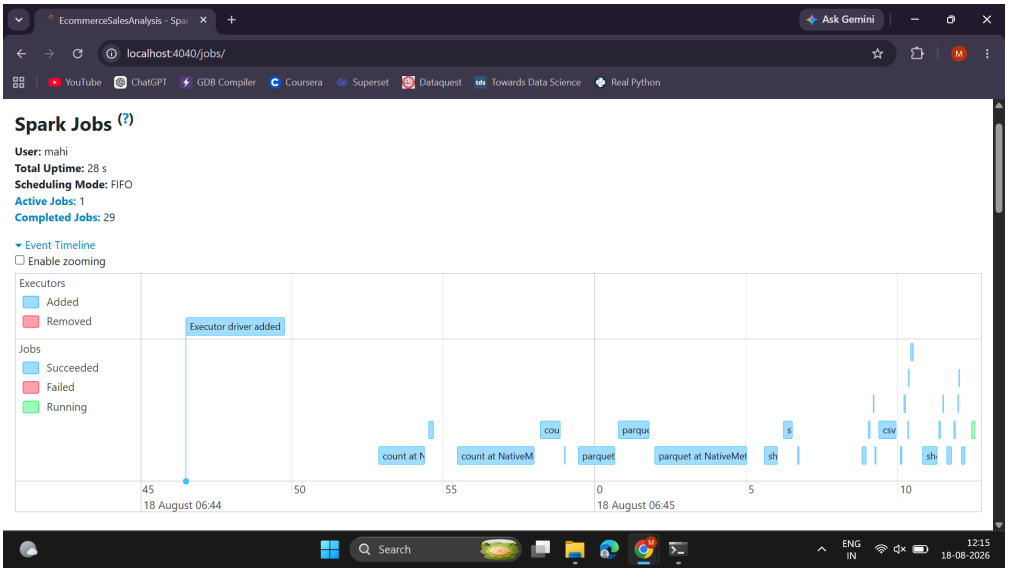

Figure 5.11: Spark Jobs tab event timeline — 29 completed jobs, one per DataFrame action (count, csv write, parquet write, showString) triggered during the run.

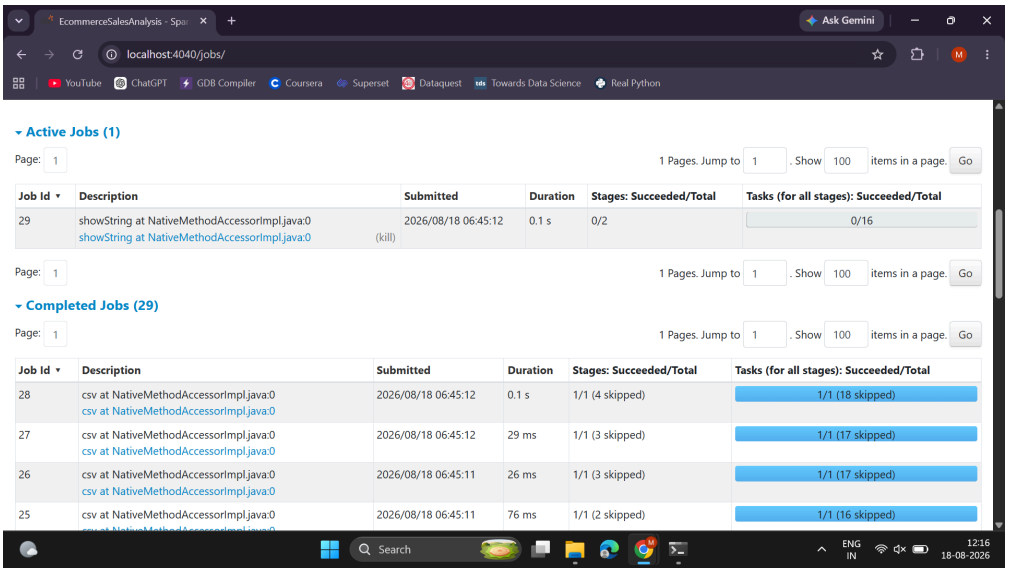

Figure 5.12: Detailed job list showing each job's description, submission time, duration, and stage/task completion — note several stages show "skipped" tasks, indicating Spark reused cached/shuffled data from earlier stages instead of recomputing it.

5.2.2 Stages Tab

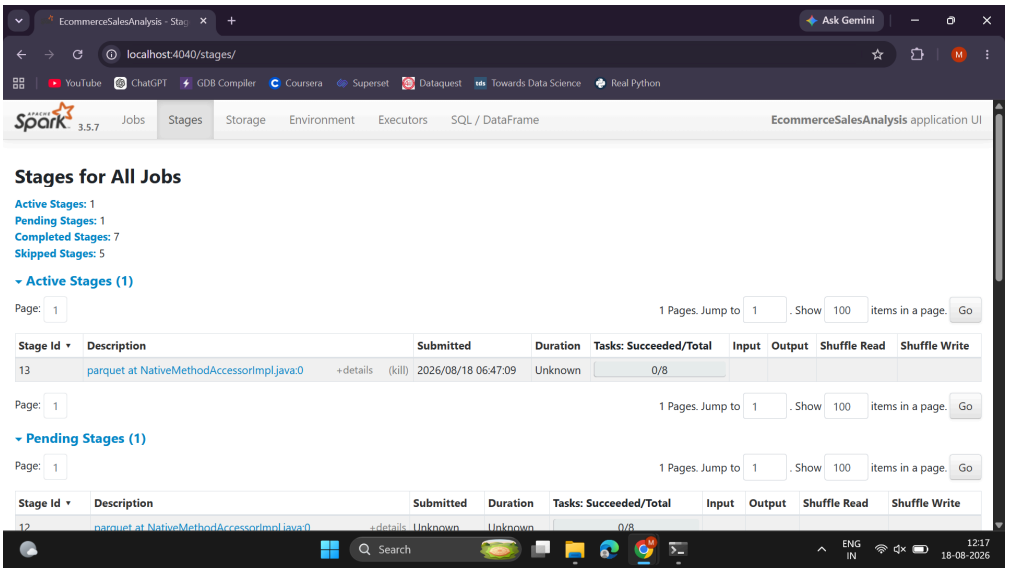

Figure 5.13: Stages overview — 7 completed, 5 skipped stages, confirming Spark's lazy evaluation and stage-reuse optimisation across the pipeline's multiple actions.

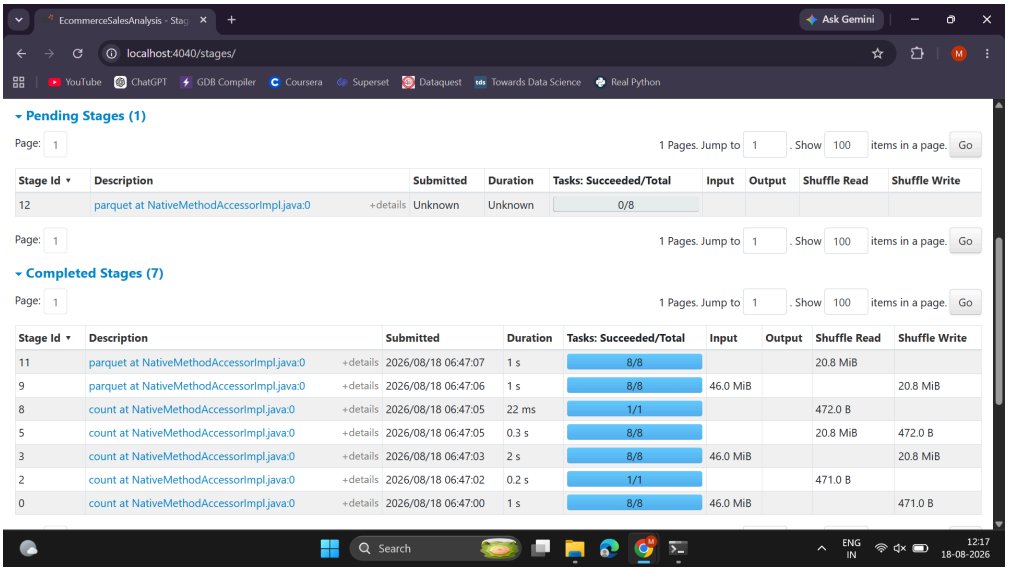

Figure 5.14: Per-stage detail showing task counts and shuffle read/write volumes (tens of KB to ~46 MiB) for each Parquet/CSV write and count operation.

5.2.3 Executors Tab

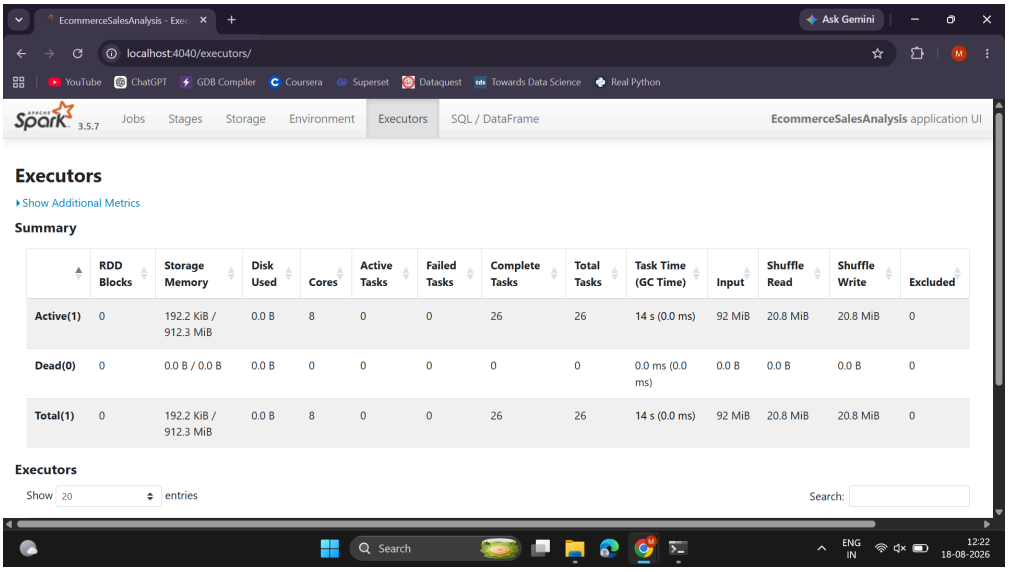

Figure 5.15: A single driver executor running with 8 cores in local[*] mode, having completed 26 tasks, processed 92 MiB of input, and shuffled ~20.8 MiB — confirming full utilisation of all available CPU cores.

5.2.4 Environment Tab

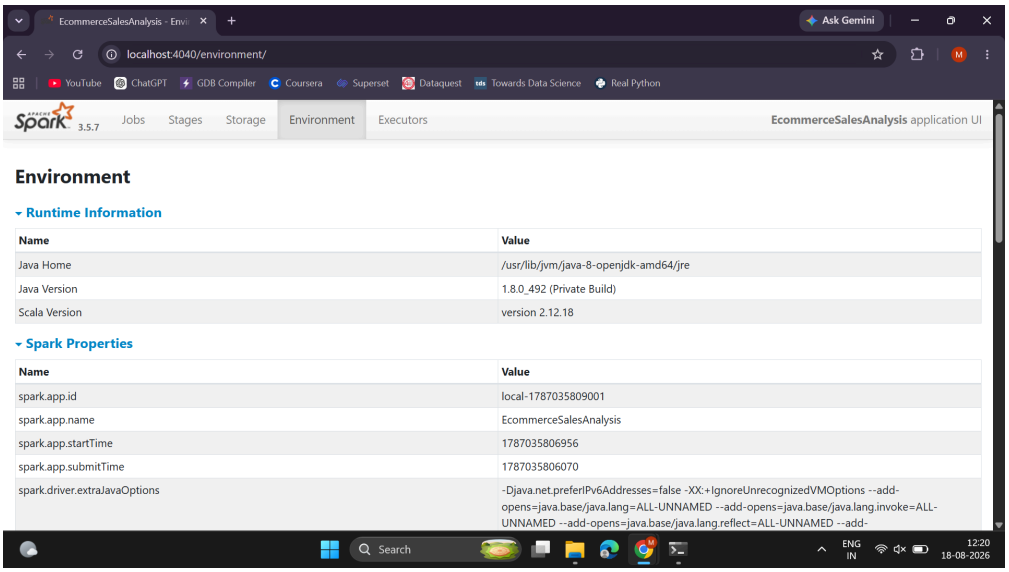

Figure 5.16: Runtime information confirming Java 8 (1.8.0_492), Scala 2.12.18, and Spark application ID/name ("EcommerceSalesAnalysis").

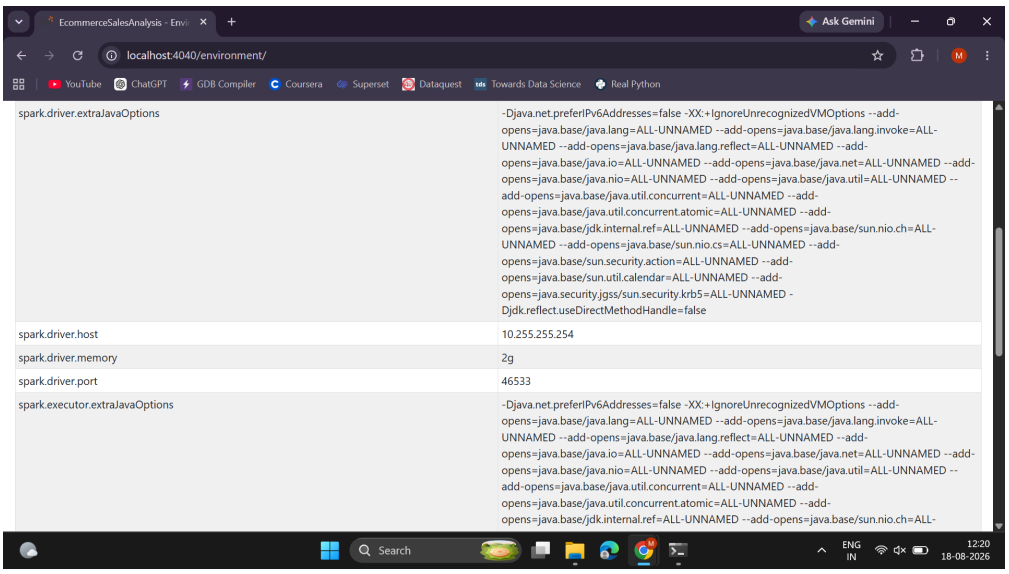

Figure 5.17: Spark configuration properties in effect for the run, including the shuffle-partition override set in code.

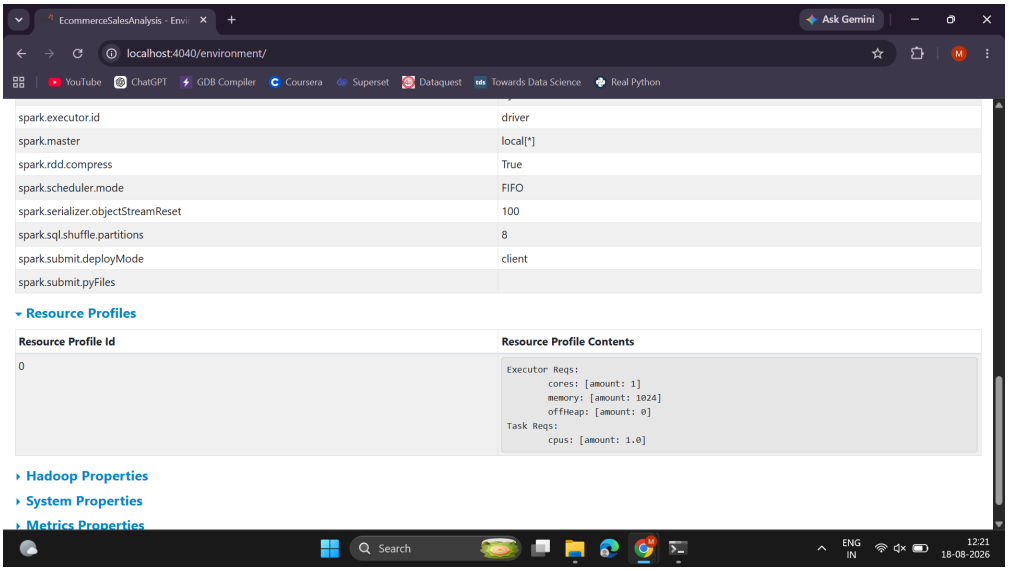

Figure 5.18: Additional system/classpath environment properties recorded by Spark for reproducibility.

5.2.5 Storage Tab

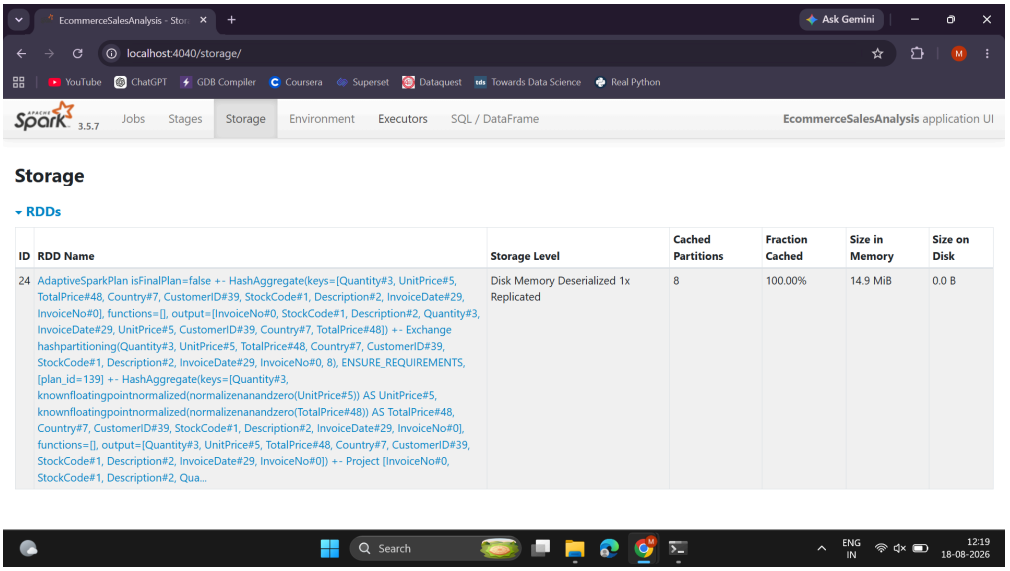

Figure 5.19: The cached cleaned DataFrame (clean_df.cache()) shown as an RDD with Disk-Memory-Deserialized storage level, 8 partitions, 100% cached, 14.9 MiB in memory — confirming the caching optimisation described in Section 3.3 took effect.

5.2.6 SQL / DataFrame Tab

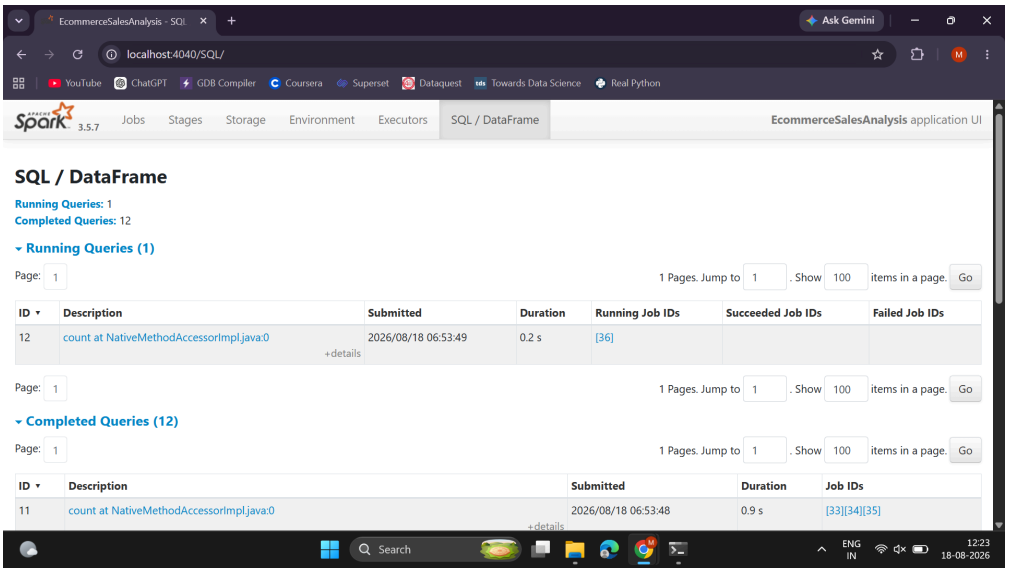

Figure 5.20: SQL/DataFrame tab listing running and completed queries, each corresponding to one Spark SQL statement or DataFrame action in the script.

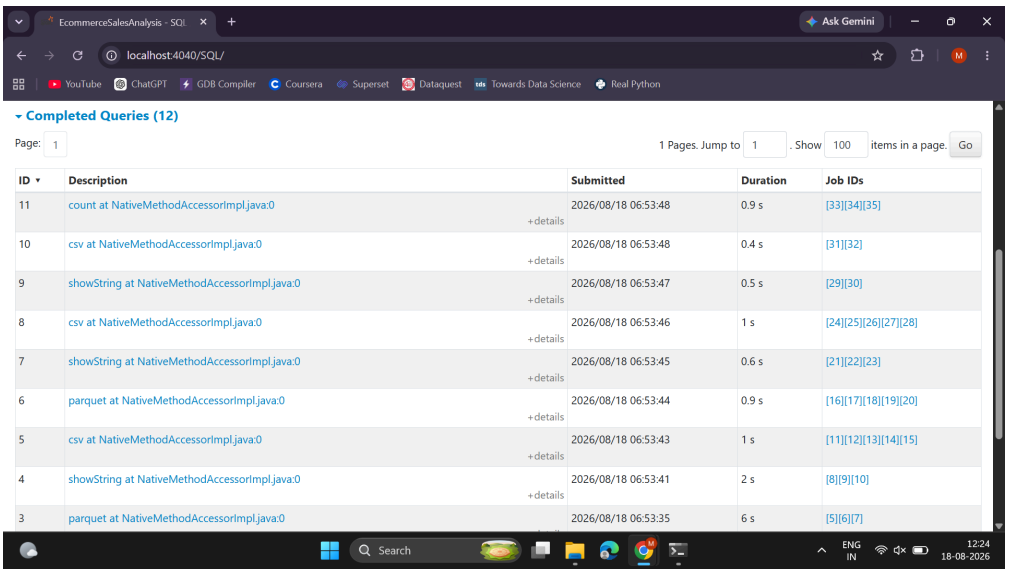

Figure 5.21: Full list of 12 completed queries with durations and linked job IDs — traceable back to the six analyses and their CSV/Parquet writes.

6. Conclusion and Future Scope

6.1 Conclusion

This project successfully implemented a complete big data pipeline for e-commerce sales analysis using Hadoop HDFS and Apache Spark on a single-node WSL2 cluster. The 541,909-row UCI Online Retail dataset was ingested into HDFS, cleaned with PySpark (removing 27.5% invalid records), and analysed using Spark SQL to answer concrete business questions — revenue concentration by country, seasonal sales trends, best-selling products, order cancellation rate, and RFM-based customer segmentation. All results were persisted back to HDFS in both CSV and Parquet formats and visualised with Matplotlib.

The analysis revealed that the business is UK-concentrated but has meaningful export markets, that sales peak sharply in November ahead of the holiday season, that volume leaders and revenue leaders among products differ, that the 14.81% cancellation rate merits further investigation, and that RFM segmentation identifies both a strong loyal customer base and a significant at-risk segment worth targeted retention efforts. Beyond the business findings, the project also demonstrates the mechanics of distributed computing in practice — the Spark UI screenshots confirm parallel task execution across 8 cores, in-memory caching of the cleaned dataset, and stage-skipping optimisations from Spark's Catalyst query planner.

6.2 Limitations

- The cluster is single-node (pseudo-distributed), so true multi-node distribution and fault tolerance were not demonstrated, only configured.

- The dataset covers a single year (Dec 2010–Dec 2011) from one UK-based retailer, so seasonal and geographic patterns may not generalise to other businesses.
- RFM scoring uses fixed quintile thresholds, which can misclassify high-value, low-frequency customers (e.g. bulk buyers), as observed with Customer 16446 in Section 4.7.

6.3 Future Scope

- Deploy on a genuine multi-node Hadoop/Spark cluster (e.g. using cloud VMs) to demonstrate real horizontal scalability and fault tolerance.
- Extend the pipeline to streaming ingestion using Kafka and Spark Structured Streaming for near-real-time sales dashboards.
- Register the cleaned Parquet output as external Hive tables to enable ad-hoc SQL querying by business analysts.
- Build an interactive dashboard (e.g. Apache Superset or Power BI) on top of the HDFS output for self-service exploration instead of static charts.
- Apply machine learning to the RFM feature set — for example, a clustering algorithm (k-means) for data-driven segment discovery instead of fixed quintile rules, or a churn-prediction classifier trained on Recency/Frequency/Monetary features.
- Extend cancellation-rate analysis to identify root causes by product, customer segment, or time period.